The Engineers AI Can’t Replace

Why Product Thinking Makes Engineers More AI-Resilient

The story of 2026

AI is getting better at an incredible rate. It can write more code, faster, and cheaper than I can. Code production is becoming a commodity.

If AI can do more of my job, what happens to me?

If code becomes commoditized, is my value becoming commoditized?

Great engineering hasn’t changed

AI simply exposed our misconceptions about how it’s been defined

Agenda

Expose

AI amplifies the existing poor definitions of engineering value

Rediscover

The skills that turn engineering work into meaningful outcomes

Apply

Put your judgment to work, recognize your impact, and communicate the value you bring

Commoditization of code

Exposed

Historically, producing software was expensive

Humans not only had to understand a problem, they also had to discover a reasonable solution, and then write it all by hand.

⬆124%

iOS app releases[1]

Writing code is cheaper than ever.

Does more code
=
better engineer?

We can generate absurd quantities of code now. Does that make us all amazing - or useless?

LOCs

Rewards volume rather than simplicity, maintainability, or correctness.

Commits & PRs

Measures units of submitted work, which vary enormously in scope and impact.

Tickets closed

Rewards completing predefined work rather than questioning whether the work should exist.

Story points

Estimates relative effort or complexity, not value delivered. They aren't standardized across teams.

We've struggled with measuring engineering productivity for decades.

Commoditization of code

We already knew output was a bad measurement

“[Output] is the most straightforward way to both understand and collect [information] from a data perspective. Unfortunately, having this as our primary focus is a flawed approach.”

- GitHub

Signals of activity, not productivity

Let’s hop on a call.

AI is amplifying output. Not necessarily value.

When code was expensive to produce, output was at least constrained by human effort. Now that AI can produce it in enormous quantities, the gap between output and value becomes even more obvious.

So what makes a good engineer?

If it’s not how much we produce, what is it?

Agenda

Exposed

AI amplifies the existing poor definitions of engineering value

Rediscover

The skills that turn engineering work into meaningful outcomes

Apply

Put your judgment to work, recognize your impact, and communicate the value you bring

Understanding value

Rediscover

How organizations view value

Revenue

Grow the business

Cost

Operate more efficiently

Risk

Reduce exposure and uncertainty

Retention

Keep customers and their trust

Reliability

Deliver a dependable experience

Opportunity

Enable future growth

Engineering value, impact, and leverage, some key definitions.

Value

What matters to the organization

Impact

The change your work creates

Leverage

How effectively your decisions and effort create impact

How engineers contribute to organizational value

Knowledge

What you've learned and understand

Experience

What you've encountered and practiced

Judgement

How you decide what matters

How do you turn your contributions into value?

It's how we apply them that creates leverage.

Solve problems

Challenge assumptions and identify the real problem.

Understand the business

Make decisions aligned with meaningful outcomes.

Think in systems

Consider technical, human, and operational dependencies.

Own the outcome

Follow the impact of your work beyond release.

Four ways to create engineering leverage

Using an example: We’re building a bill pay feature for our banking app.

Seems simple enough. Let’s use this feature to explore how engineers can leverage knowledge, experience, and judgement to deliver the most impact.

Solve problems.
not tickets.

What does that mean?

Aren’t I already solving problems?

A ticket isn’t a problem. It’s someone else’s thoughts encoded through their ability to express it in words

The author injected their assumptions, partial solutions, pre-explored constraints. Starting from a ticket can remove your ability to critically think about the problem being solved.

85%

Agreed their organizations were poor at diagnosing problems[2], even more agreed this weakness carried significant costs.

Build recurring bill payments so customers can automatically pay their bills every month.

Bill pay

Customers can reliably pay their bills with less effort

Understand the desired outcome

Work backwards.

Challenge the proposed solution, understand the underlying problem, and work backward from the desired outcome.

Solve problems

Challenge assumptions and identify the real problem.

Understand the business

Make decisions aligned with meaningful outcomes.

Think in systems

Consider technical, human, and operational dependencies.

Own the outcome

Follow the impact of your work beyond release.

Four ways to create engineering leverage

Understanding value

Understand the business

Let’s be real - we all want to keep getting paid.

A successful user or business outcome is the reason you continue to have a job

Understanding where your organization is headed will help you make decisions more aligned to the expectations of leadership - and ideally your customers.

40%

Teams that focus on the user have higher organizational performance.[3] Correlating a greater understanding of the intended outcomes with stronger technical decisions.

Customers can reliably pay their bills with less effort.

Our solution could be:

Build everything

High investment - broad functionality

Integrate a 3rd party

Medium investment - faster capability

Start narrow

Low initial investment - validate adoption

Business context changes engineering decisions

Different business priorities can have an impact on the same types of problems

Solve problems

Challenge assumptions and identify the real problem.

Understand the business

Make decisions aligned with meaningful outcomes.

Think in systems

Consider technical, human, and operational dependencies.

Own the outcome

Follow the impact of your work beyond release.

Four ways to create engineering leverage

Think in systems

Treat everything like a system, even your organization

Software doesn’t exist in isolation

Technical

Services, APIs, infrastructure, data, etc...

Human

Teams, ownership, communication decisions

Operational

Processes, handoffs, approvals, support

Bill pay is more than just a set of APIs

Is the work done if it’s technically complete?

Your responsibility is what your build.

You enable better decisions, coordination, and processes.

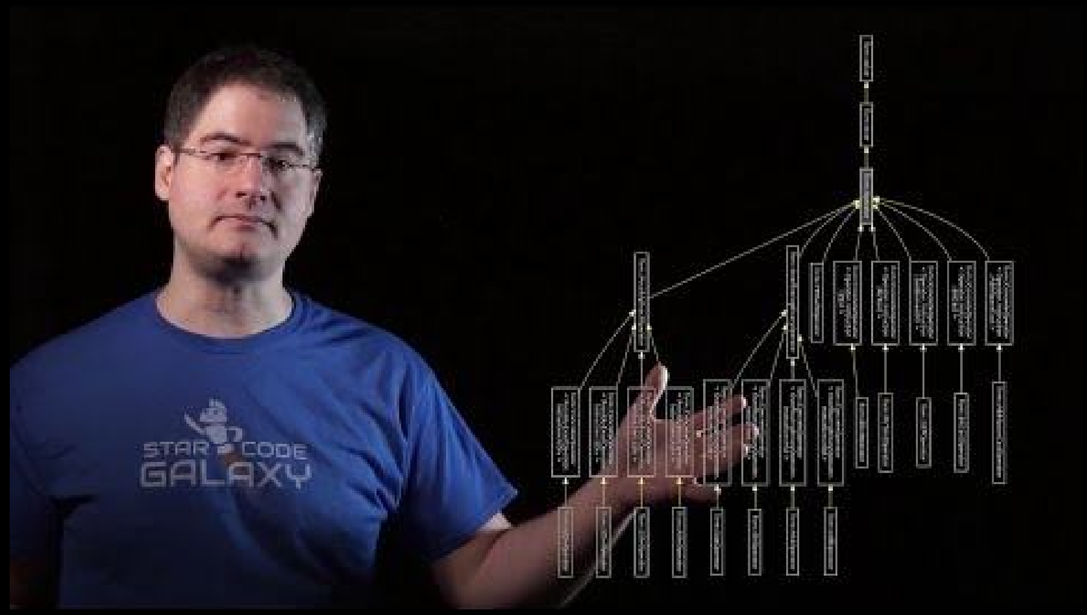
Productivity extends beyond individual activity[4]

Collaboration and coordination affect engineering productivity, even when those contributions don't produce easily counted artifacts.

Think about the most impactful changes on your teams, have they come from code?

Any time you have an reorg, it’s massively disruptive. The impact on performance of your entire company, your team, and the operations you perform are clearly felt.

Conway’s law[5]



Your judgment can improve systems beyond the work you're assigned.

Your work, and the work of others, is affected by systems you don't own.

Solve problems

Challenge assumptions and identify the real problem.

Understand the business

Make decisions aligned with meaningful outcomes.

Think in systems

Consider technical, human, and operational dependencies.

Own the outcome

Follow the impact of your work beyond release.

Four ways to create engineering leverage

Your job extends beyond your artifacts

Ticket moved to released. Is your responsibility over?

We launched bill pay! Did it work?

Adoption

How many eligible customers use bill pay?

Completion

How many customers are utilizing the flow?

Reliability

Were there bugs or errors? How many? Why?

Customer impact

Does bill pay reduce effort or increase retention?

How can we tell if a system isn’t working? In software, it’s obvious.

But it my not be as clear in your organizational systems. Data collection and continuity, the ability to observe and measure outcomes of your team, product, and company will all help improve your teams ability to make good decisions.

The cost of decision making

↑ 5-6%

Productivity associated with data-driven decision-making.[6]

-$12.9M/year

Average estimated organizational cost of poor data quality.[7]

Close the loop

Define success. Observe the outcome. Learn from the result.

Agenda

Exposed

AI amplifies the existing poor definitions of engineering value

Rediscover

The skills that turn engineering work into meaningful outcomes

Apply

Put your judgment to work, recognize your impact, and communicate the value you bring

Communicate your value

Apply

Think about the best engineer you know

What makes them stand out?

What makes us valuable?

Beyond the code we’ve written

LOCs

Rewards volume rather than simplicity, maintainability, or correctness.

Commits & PRs

Measures units of submitted work, which vary enormously in scope and impact.

Tickets closed

Rewards completing predefined work rather than questioning whether the work should exist.

Story points

Estimates relative effort or complexity, not value delivered. They aren't standardized across teams.

We were already confusing activity with impact

What you bring to the next opportunity

Knowledge

What you've learned and understand

Experience

What you've encountered and practiced

Judgement

How you decide what matters

Solve problems

Am I solving the right problems?

Understand the business

Does this align with what the business needs?

Think in systems

What other systems or people influence success?

Own the outcome

How will I know whether it worked?

The skills behind our impact

Communicate your value

Your code may become easier to reproduce.

Your accumulated judgment is still hard to replicate.

What impact will you bring to my team?

Stop defining yourself primarily through your output

Stop focusing on what you produce.

Start focusing on creating leverage.

Thank you

The Engineers AI Can’t Replace

Why Product Thinking Makes Engineers More AI-Resilient